LAB SESSION NO. 2 Part-2

PROLOG-RULES

OBJECTIVE:

To learn how to define and implement rules.

THEORY:

So far we have looked at how to represent facts and to query them. Now we move on to rules. Rules allow us to make conditional statements about our world. Each rule can have several variations, called clauses. These clauses give us different choices about how to perform inference about our world. Let's take an example to make things clearer.

Consider the following

'All men are mortal':

We can express this as the following Prolog rule.

```
mortal(X) :-

    human(X).
```

The clause can be read in two ways (called either a declarative or a procedural interpretation). The declarative interpretation is "For a given X, X is mortal if X is human." The procedural interpretation is "To prove the main goal that X is mortal, prove the subgoal that X is human."

Rules 2:

To continue our previous example, lets us define the fact 'Neuton is human' so that our program now looks as follows:

```
mortal(X) :-

human(X).

human(neuton).
```

If we now try the question to Prolog

?- mortal(neuton).

The Prolog interpreter would respond if it is written in the facts as follows:

Yes

Why was this? Well in order to solve the query ?- mortal(neuton)., we used the rule we saw previously. This said that in order to prove someone mortal, we had to prove them to be human. Thus from the goal Prolog generates the subgoal of showing human(neuton).

In the above example we were able to match human(neuton) against the knowledgebase described at the top of this file. In Prolog we say that the subgoal succeeded, and as a result the overall goal succeeded. We know when this happens because Prolog prints yes.

Rules 3

We can also use variables within queries. For example, we might wish to see if there is somebody who is mortal. This is done by the following line.

?- mortal(P).

The Prolog interpreter responds.

P = neuton

Yes

Rule- Clause that depend on other facts

Example:

Facts of the rule as follows:

toy(bear).

toy(snoopy).

toy(car).

play(saif,snoopy).

play(ali,tweety).

play(saif,bear).

```
like (A,B):-
        toy(B),
        play(A,B).
```

After Compile the query results will be as follows:

```
?- like(X,Y).
X = saif ,
Y = bear ;
X = saif ,
Y = snoopy ;
no
?- like(X,_).
X = saif ;
X = saif ;
no
?- like(_,X).
X = bear ;
X = snoopy ;
no
```

Question: Why ali, car and tweety is not going to show in the query?

Ans: ……

Example:

If A is in B AND B is in C then A is in C.

in prolog:

```
is_in(A,C):-
        in(A,B),
        in(B,C).
```

Note: Add facts by yourself and try to generate your own query for the above rule and check out the results which prolog will give you according to your facts.

Example:

If A is clever OR A is smart THEN A is intelligent.

in prolog:

```
intelligent(A):-
        clever(A);
        smart(A).
```

Note: Add facts by yourself and try to generate your own query for the above rule and check out the results which prolog will give you according to your facts.

This means that Prolog was able to prove the goal by binding the variable P to neuton. This was done by again proving someone was mortal by proving the subgoal that they were human. Prolog thus asked if there was any P that was human. This matches against the clause human(neuton) thereby binding P to neuton. This binding is then passed back to the parent goal, and the results in the printout we saw above.

Rules 4

Sometimes we may wish to specify alternative ways of proving a particular thing. This we can do by using different rules and facts with the same name. For example, we can represent the sentence 'Something is fun if its a red car or a blue bike or it is ice cream' as follows:

```
fun(X) :-                /* an item is fun if */

    red(X),              /* the item is red */

    car(X).              /* and it is a car */

fun(X) :-                /*  or an item is fun if */

    blue(X),             /* the item is blue */

    bike(X).             /* and it is a bike */

fun(ice_cream).              /* and ice cream is also fun. */
```

This program says that we have three ways of finding out if something is fun. Something is fun if it is a red and a car or blue and a bike, or if it is ice cream. These three options are represent in Prolog by three clauses of the predicate fun. Just like we saw for pure facts, Prolog will start from the first clause (be it a rule or fact) of fun and try that. If that does not succeed, we try the next clause. We only fail when we run out of rules or facts to try.

Rules 5

All identically-named variables within a particular rule (e.g. all occurrences of, say, X in the first fun rule below) are constrained to have one and the same instantiation for each solution to a particular query. Identical variable names in separate rules are totally independent, just as if different variable names had been used. Because the consequences of the preceding two sentences are so fundamental to Prolog (and so often misunderstood), we recommend that you read them again. To give an example

The program

```
fun(X) :-

        red(X),

        car(X).

fun(X) :-

        blue(X),
```

```
        bike(X).
```

<u>Note:</u> Looks to Prolog Like

```
fun(X_1) :-
        red(X_1),
        car(X_1).
fun(X_2) :-
        blue(X_2),
        bike(X_2).
```

Thus variable name scoping is per-individual rule (often called a clause). The same variable name may appear in different clauses of a rule, or rules with different names. Each time it is treated as something specific to its context. A variable X may occur in the program many times with many different bindings. It will only have the same bindings when you tell it to.

Examples of Rules 1

Consider the following program:

```
fun(X) :-
        red(X),
        car(X).
fun(X) :-
        blue(X),
        bike(X).
car(vw_beatle).
car(ford_escort).
bike(harley_davidson).
red(vw_beatle).
red(ford_escort).
```

```
blue(harley_davidson).
```

Above is both our previous program for finding fun items and facts describing some red and blue items and some cars and bikes. Let's now use the above program and see if a harley_davidson is fun. To do this we can ask Prolog the following question.

```
?- fun(harley_davidson).          /* to which Prolog will reply */

yes                               /* to show the program succeeded */
```

To execute this query, Prolog will first see if harley_davidson is red, however only vw_beatles and ford_escorts are defined as being red. Hence the query red(harley_davidson) will fail. This means that the first clause of fun will fail. As a result, we now try the second clause of fun. This will mean that we will attempt the subgoals blue(harley_davidson) and bike(harley_davidson). Both these goals match facts in the knowledgebase. As a result fun succeeds as we saw above.

Examples of Rules 2

```
fun(X) :-
        red(X),
        car(X).
fun(X) :-
        blue(X),
        bike(X).
car(vw_beatle).
car(ford_escort).
bike(harley_davidson).
red(vw_beatle).
red(ford_escort).
blue(harley_davidson).
```

We can also ask our program to find fun items for us. To do this we can pose the following question.

?- fun(What).

To which Prolog will reply

What=vw_beatle yes

Let's see how Prolog deals with this query. Firstly we will try the first clause of fun. This results in us trying the goal red(What). This succeeds matching the first clause of red with the binding What=vw_beatle. Now we attempt the goal car(vw_beatle). This matches the first clause of car, and, as a result, the fun goal succeeds.

Rules Exercises

<u>Exercise 1</u>

Indicate whether the following are syntactically correct Rules. **Write answer Yes or No.**

a :- b, c, d:- e f. Is this syntactically correct?

happy(X):- a , b. Is this syntactically correct?

happy(X):- hasmoney(X) & has_friends(X). Is this syntactically correct?

fun(fish):- blue(betty), bike(yamaha). Is this syntactically correct?

<u>Exercise 2</u>

Given the database below, study the queries underneath it. Indicate whether you think a particular query will succeed or fail by **answer yes or no.**

```
likes(john,mary).

likes(john,trains).

likes(peter,fast_cars).

likes(Person1,Person2):-

        hobby(Person1,Hobby),

        hobby(Person2,Hobby).

hobby(john,trainspotting).
```

hobby(tim,sailing).

hobby(helen,trainspotting).

hobby(simon,sailing).

?- likes(john,trains). Will this Succeed?

?- likes(helen,john). Will this Succeed?

?- likes(tim,helen). Will this Succeed?

?- likes(john,helen). Will this Succeed?

Exercise 3:

How many facts, rules, clauses, and predicates are there in the following knowledge base? What are the heads of the rules, and what are the goals they contain?

woman(nimmy).

woman(maria).

man(jabbaar).

person(X) :- man(X); woman(X).

loves(X,Y) :- knows(Y,X).

father(Y,Z) :- man(Y), son(Z,Y).

father(Y,Z) :- man(Y), daughter(Z,Y).

Exercise 4:

Represent the following in Prolog:

- Bashir is a killer.
- Maria and Maqbool are married.
- Zaib is dead.
- Maqbool kills everyone who gives Maria a foot massage.
- Maria loves everyone who is a good dancer.
- Jabbaar eats anything that is nutritious or tasty.

Exercise 5:

Suppose we are working with the following knowledge base:

```
wizard(ronny).

hasStick(harry).

footballPlayer(harry).

wizard(X) :- hasBroom(X),hasStick(X).

hasBroom(X) :- footbalPlayer(X).
```

How does Prolog respond to the following queries?

- wizard(ronny).
- witch(ronny).
- wizard(amar).
- witch(amar).
- wizard(harry).
- wizard(Y).
- witch(Y).

Some more examples that can help you to understand prolog rule without using variable as follows:

read() is a keyword/builtin function that can read the input from user/keyboard.

Program1:

```
start:- sum,nl.

sum:-   write('X= '),read(X),

        write('Y= '),read(Y),

        S is X+Y,

        write('Sum is '),write(S).
```

Output:

```
?- start.

X= |: 1.

Y= |: 2.

Sum is 3

yes
```

Program 2:

```
double:- write('Please input a number= '),
                read(X),nl,
                write('Double is '), Y is 2*X,
                write(Y),nl.
```

Output:

```
?- double.

Please input a number=  2.

Double is 4

yes
```

Program 3:

```
start:- write('X= '),read(X),
        write('Y= '),read(Y),
        R is (X+Y)/2, //calculating mean of given 2 numbers
        write('R is '),write(R),nl.
```

Output:

?- start.

X= 4.

Y= 6.

R is 5

yes

Lab EXERCISE:

Suppose that each member of a club is associated with a record of information. For example:

- **Name:** **Asgar Ali**
- **Gender:** **male**
- **Heights (in Feet):** **5.4**
- **Weight (in kg) :** **65**
- **Hobbies:** **reading, singing, game_playing**

1. Write a Prolog database containing data on the club's members by using rules to facilitate retrieval of the following information:

a. **Given a name, find height or weight or hobbies**
b. **Find someone with a particular hobby.**
c. **Show all the male members from facts by using backtracking method.**

Note: For Each part of above task should take value from user individually.

2. Write a prolog program that will get value from user and show it's previous and next value in sequence as in given output.

Output:

?- value_p_n(P,5,N).

P = 4

N = 6

After getting the above result from prolog code then change in code to take value from user such as:

?- start.

Type any integer number: 2.

Previous number is: 1.

Next number is: 3.